GOAL FIRST — TEIL ZWEI

GOAL FIRST IN PYTHON

BAU DIR DEINEN EIGENEN AKTIEN-SCREENER

Ein Framework, das deine Anlagelogik in Code übersetzt.

2

Leverage Thinking · Laura Kruse-Stecher

Inhalt

Einleitung
Das Framework, und wie du diesen Guide nutzt

Teil 1 — Von der Logik zum Code
Pseudocode vor Python

Teil 2 — Datenqualität
Drei Probleme, die du vor dem Screening abfängst

Teil 3 — Der Screener
Kernfunktion und Ausgabe

Teil 4 — Metriken ergänzen
Drei Kennzahlen, die zählen

Teil 5 — An dein Ziel anpassen
Varianten Dividende und Low Volatility

Fazit
Einmal bauen, immer nutzen

Glossar
Über die Autorin · Code & Updates · Impressum

Disclaimer

Keine Anlageberatung

Diese Publikation dient ausschließlich Informations- und Bildungszwecken. Sie ist keine Anlageberatung und keine Finanzanalyse im Sinne des Wertpapierhandelsgesetzes, keine Empfehlung zum Kauf oder Verkauf eines Wertpapiers und keine Aufforderung irgendeiner Art. Sie stellt eine Methode vor; Anlageentscheidungen triffst du selbst.

Eigenverantwortung

Alle Anlageentscheidungen liegen allein in deiner Verantwortung. Die in diesem Guide beschriebene Kapitalallokation, Strategieauswahl und Gewichtung von Portfoliobestandteilen sind illustrative Rahmen – keine Vorgaben. Deine finanzielle Situation, deine Risikotoleranz und deine Anlageziele sind individuell und können von jedem hier gezeigten Beispiel erheblich abweichen.

Risikohinweis

Aktienanlagen sind mit Risiken verbunden, bis hin zum Verlust des eingesetzten Kapitals. Vergangene Wertentwicklung – einschließlich aller historischen Beispiele in diesem Guide – ist kein Indikator für zukünftige Ergebnisse.

Beispiele und Daten

Die in diesem Guide genannten Unternehmen, darunter AT&T und Snap, dienen ausschließlich als Fallbeispiele zur Veranschaulichung methodischer Prinzipien. Sie stellen keine aktuellen Empfehlungen oder Meinungen zu diesen Wertpapieren dar. Kurs- und Renditeangaben sind eigene Berechnungen auf Basis öffentlich verfügbarer Kursdaten, gerundet; Stand bei Erstellung des Guides. Abweichungen je nach Datenquelle und Berechnungsmethode sind möglich.

Eigene Positionen

Die Autorin handelt selbst Aktien und kann in einzelnen der genannten Werte investiert sein oder gewesen sein. Die Auswahl der Beispiele erfolgte nach ihrer Eignung zur Veranschaulichung, nicht nach eigenen Positionen.

Software und Daten

Die Codebeispiele in diesem Guide sind Werkzeuge zur Datenverarbeitung und Analyse. Die Richtigkeit jeder Ausgabe hängt von Qualität und Aktualität der zugrunde liegenden Datenquellen ab. Die Beispiele verwenden die Python-Bibliothek yfinance, die Daten von Yahoo Finance abruft. yfinance ist ein unabhängiges Open-Source-Projekt und weder von Yahoo herausgegeben noch mit Yahoo verbunden; die Nutzung der abgerufenen Daten unterliegt den Nutzungsbedingungen von Yahoo. Informiere dich vor dem Einsatz über die jeweils geltenden Bedingungen – insbesondere, wenn du Ergebnisse weitergeben, veröffentlichen oder geschäftlich nutzen willst. Der Code in diesem Guide steht unter der MIT-Lizenz; die Lizenz liegt im Repository (Seite „Code & Updates").

Haftung

Die Autorin übernimmt, soweit gesetzlich zulässig, keine Haftung für Entscheidungen, die auf Grundlage dieses Guides oder der darin enthaltenen Beispiele, Daten und Codebeispiele getroffen werden.

Angaben zur Autorin

Die Autorin ist in keiner Jurisdiktion als Finanzberaterin, Anlageberaterin oder Broker-Dealer zugelassen.

Einleitung

Die Goal-First-Methode funktioniert auf dem Papier. Irgendwann hört sie auf zu skalieren.

Zwanzig Jahre Renditedaten für 50 Ticker von Hand ziehen, jeden Jahr für Jahr gegen eine Benchmark prüfen, nach Verlustjahren filtern, nach Durchschnittsrendite rangieren – beim ersten Mal ist das befriedigend. Bei der dritten Iteration kopierst du um Mitternacht Zahlen in Tabellen und stellst deine Lebensentscheidungen infrage.

Python löst das. Nicht weil es Magie wäre, sondern weil es schnell ist, wiederholbar, und – sobald die Logik steht – überraschend unkompliziert. Wenn du deine Anlagekriterien klar genug beschreiben kannst, um sie einer anderen Person zu erklären, kannst du sie in einen Screener verwandeln. Mehr braucht es tatsächlich nicht.

Der Ausgangspunkt ist nicht der Code. Es ist dieselbe Frage wie vorher: Wofür wählst du aus, und was schließt einen Kandidaten aus? Sobald das klar ist, ist die Übersetzung nach Python fast mechanisch. Der Code in diesem Guide folgt dem Denken – nicht umgekehrt.

> *Du musst kein Entwickler sein, um etwas Nützliches zu bauen. Du musst wissen, was du willst.*

Wenn du Goal First – Erst das Ziel, dann die Aktien gelesen hast, kennst du das Framework bereits. Dieser Guide ist seine Python-Implementierung. Wenn nicht, ist der erste Guide der bessere Einstieg.

Noch etwas: Wenn du das lieber gebaut bekommst – angepasst an deine Ziele, deine Datenquelle, deine Kennzahlen –, melde dich über Gumroad.

Teil 1

Von der Logik zum Code

Das abstrakte Muster

Jedes Screening-Framework, unabhängig von der Strategie, hat dieselbe dreiteilige Struktur. Ziel, Constraints, Ranking. Mehr nicht.

```
DEFINE goal          -> the metric you rank candidates by
DEFINE constraints   -> conditions that disqualify a candidate
RANK                 -> order what remains by the goal metric
```

Als Pseudocode – eine Ebene näher am echten Code, aber noch als reine Logik lesbar:

```
FOR each ticker in universe:
    metric = calculate_goal_metric(ticker)

    FOR each constraint in constraints:
        IF constraint is violated:
            EXCLUDE ticker
            BREAK

    IF ticker not excluded:
        ADD to candidates with metric

RANK candidates BY metric DESCENDING
RETURN candidates
```

Diese Struktur ändert sich zwischen Strategien nicht. Was sich ändert, sind die Eingaben: welche Kennzahl das Ziel definiert und welche Bedingungen zum Ausschluss führen. Die Python-Implementierung bildet das direkt ab – eine Kern-Screening-Funktion, Parameter, die du kontrollierst.

> *Der Vorteil, hier anzufangen: Hast du das einmal gebaut, ist die Anpassung an eine andere Strategie eine Sache von ein paar Zeilen – kein Neuschreiben.*

Deine Parameter festlegen

Bevor du eine einzige Funktion schreibst, beantworte drei Fragen. Schreib sie auf. Das ist die Spezifikation. Alles andere ist Ausführung.

WAS IST DEIN ZIEL?

Die Kennzahl, nach der du Kandidaten rangierst – oder, wie bei der Outperformance-Strategie, die Eigenschaft, die ein Kandidat mindestens erfüllen muss, bevor rangiert wird. Benchmark-Trefferquote, Dividendenrendite, Free-Cashflow-Rendite. Eine Kennzahl. Hast du zwei, entscheide, welche zuerst kommt. Der Screener kann nur nach einer Sache rangieren – dieser Zwang erzeugt eine Klarheit, die die meisten manuellen Ansätze stillschweigend vermeiden.

WAS SIND DEINE CONSTRAINTS?

Bedingungen, die einen Kandidaten disqualifizieren, egal wie gut er bei der Zielkennzahl abschneidet. Binär – bestanden oder nicht, keine Teilpunkte. Jeder Constraint sollte vor einem bestimmten Fehlertyp schützen. Wenn du nicht erklären kannst, warum ein Constraint da ist, sollte er wahrscheinlich nicht da sein.

WAS IST DEIN UNIVERSUM?

Die Liste der Ticker, die du screenst. Ein Universum können die S&P-500-Mitglieder sein, die großen Namen eines Sektors oder eine eigene Watchlist. Garbage in, garbage out: Ein gut gebauter Screener auf einem schlecht gewählten Universum liefert trotzdem schlechte Ergebnisse.

Parameter	Outperformance	Dividende
Zielvariable	Benchmark-Trefferquote ≥ 0,60	Dividendenrendite ≥ 3,5 %
Constraint 1	Verlustjahrquote ≤ 0,25	Verlustjahrquote ≤ 0,25
Constraint 2	Mind. 10 Jahre Historie	Ausschüttungsquote ≤ 90 %
Constraint 3	—	Mind. 10 Jahre Historie
Ranking	Ø Jahresrendite, absteigend	Ø Gesamtrendite, absteigend
Universum	S&P-500-Mitglieder	Eigene Dividenden-Watchlist

Beide Beispielstrategien verwenden die Verlustjahrquote als ersten Constraint. Das ist eine Wahl, keine Vorgabe: Constraints werden je Strategie gewählt. In den gezeigten Beispielen ist die Verlustjahrquote der Filter, der zuerst greift, weil er den häufigsten Fehlertyp abfängt – eine Aktie, deren Zielkennzahl gut aussieht, die aber regelmäßig Geld verliert.

Zwei Beispiele in Pseudocode

Beispiel 1 — Outperformance

Ziel: Aktien, die den S&P 500 konsistent schlagen – rangiert nach durchschnittlicher Jahresrendite.

```
FOR each ticker in universe:
    annual_returns    = get_annual_returns(ticker, years=20)
    benchmark_returns = get_annual_returns('^GSPC', years=20)

    # Goal: must beat benchmark consistently
    bias_score = count(annual_returns > benchmark_returns) / years
    IF bias_score < 0.60: EXCLUDE

    # Constraint 1: loss year ratio within personal threshold
    loss_year_ratio = count(annual_returns < 0) / years
    IF loss_year_ratio > max_loss_ratio: EXCLUDE

    # Ranking metric: average annual return (arithmetic mean)
    avg_return = mean(annual_returns)
    ADD to candidates

RANK BY avg_return DESCENDING
```

Die Trefferquote von 0,60 heißt: Die Aktie hat den S&P 500 in mindestens 60 % der beobachteten Jahre geschlagen. Die Schwelle für die Verlustjahrquote ist persönlich, nicht fix. Ein vernünftiger Standard für die meisten Anleger ist 0,25: War eine Aktie in mehr als einem Viertel aller beobachteten Jahre negativ, besteht sie nicht. Beides sind deine Parameter, keine Gebote.

Beispiel 2 — Dividende

Ziel: Aktien mit einer tragfähigen Dividendenrendite, rangiert nach durchschnittlicher Gesamtrendite.

```
FOR each ticker in universe:
    div_yield      = get_dividend_yield(ticker)
    annual_returns = get_annual_returns(ticker, years=20)
    payout_ratio   = get_payout_ratio(ticker)

    # Constraint 1: loss year ratio within personal threshold
    loss_year_ratio = count(annual_returns < 0) / years
    IF loss_year_ratio > max_loss_ratio: EXCLUDE

    # Constraint 2: dividend must come from earnings
    IF payout_ratio > 0.90: EXCLUDE

    # Goal: yield above threshold
    IF div_yield < min_yield: EXCLUDE

    # Ranking metric: average total return (arithmetic mean)
    avg_return = mean(annual_returns)
    ADD to candidates

RANK BY avg_return DESCENDING
```

Gleiche Struktur, andere Kennzahlen. Der Ausschüttungsquoten-Constraint fängt das AT&T-Muster aus dem ersten Guide ab – eine Rendite, die attraktiv aussieht, aber aus etwas anderem als stabilen Gewinnen finanziert wird. Ein Constraint, ein Fehlertyp eliminiert.

Warum das vor dem Code zählt

Den Pseudocode oben könnte jeder schreiben, der eine klare Anlagethese und eine ruhige halbe Stunde hat. Das ist der Punkt. Das Python, das folgt, ist eine direkte Übersetzung dieser Logik – keine architektonischen Überraschungen, keine versteckten Annahmen.

Ist der Pseudocode falsch, ist der Code auf genau dieselbe Weise falsch. Das ist tatsächlich ein Vorteil: Der Fehler ist sichtbar, behebbar und an einer Stelle, die du kontrollierst.

> *Ein Screening-Framework ist nur deine Anlagelogik in Code. Ist deine Logik klar, ist der Code einfach. Ist deine Logik unscharf, ist es der Code auch.*

Teil 2

Datenqualität

Die Screening-Logik aus Teil 1 ist nur so gut wie die Daten, auf denen sie läuft. Das ist weniger offensichtlich, als es klingt.

yfinance ist für die meisten Zwecke zuverlässig. Aber „die meisten Zwecke" leistet in diesem Satz viel Arbeit. Für ein Framework, das Entscheidungen auf Basis von 20 Jahren Jahresrenditen trifft, summieren sich kleine Datenprobleme zu großen Ausgabeproblemen – leise, ohne Fehlermeldung. Ein Screener, der wegen schlechter Daten die falschen Kandidaten liefert, ist schlimmer als gar kein Screener, weil er sich anfühlt, als würde er funktionieren.

Drei Probleme lohnt es sich abzufangen, bevor der Screener läuft: ungültige Ticker, Kursverzerrungen durch Splits und Zombie-Aktien. Jedes hat seinen eigenen Detektor. Zusammen bilden sie eine Datenqualitätsschicht zwischen deinem Universum und deiner Screening-Logik.

Und danach, nach alledem, schaust du dir die Ausgabe trotzdem selbst an. Dazu mehr am Ende.

Der Ticker-Validator

Das einfachste Problem: ein Ticker, der nicht funktioniert. Das passiert öfter als erwartet – Ticker ändern sich, wenn Unternehmen fusionieren, übernommen werden oder sich umstrukturieren. Eine Liste von Indexmitgliedern von vor sechs Monaten kann bereits veraltete Einträge enthalten.

yfinance wirft bei einem ungültigen Ticker keine Exception. Es liefert einen leeren DataFrame. Fängst du das nicht ab, überspringt der Screener die Aktie stillschweigend oder – schlimmer – produziert einen Rechenfehler, der sich nach unten durchzieht.

```
def validate_tickers(tickers: list[str],
                     min_years: int = 10) -> dict:
    """
    Checks each ticker for data availability and
    minimum history length.

    Returns:
        {
          'valid': [...],
          'invalid': [...],
          'insufficient_history': [...]
        }
    """
```

Die Funktion liefert drei Gruppen: gültige Ticker mit ausreichender Historie, Ticker, die gar keine Daten liefern, und Ticker, bei denen Daten existieren, aber den Mindestzeitraum nicht

abdecken. Eine Aktie mit nur 3 Jahren Historie lässt sich in einem 20-Jahres-Framework nicht bewerten.

Ticker	Status	Verfügbare Jahre
Aktie A	✓ Gültig	22
Aktie B	✓ Gültig	18
Aktie C	✓ Gültig	20
Aktie D	× Ungültig	—
Aktie E	⚠️ Historie zu kurz	4
Aktie F	✓ Gültig	20
Aktie G	× Ungültig	—
Aktie H	✓ Gültig	15
Aktie I	⚠️ Historie zu kurz	7
Aktie J	✓ Gültig	21

Sechs gültig, zwei ungültig, zwei mit zu kurzer Historie. Das läuft vor allem anderen.

Der Split-Detektor

yfinance bereinigt historische Kurse automatisch um Aktiensplits. Meistens funktioniert das korrekt. Bei älteren Daten – besonders bei Splits vor einem bestimmten Datum oder bei weniger liquiden Aktien – kann die Bereinigung falsch sein oder ganz fehlen.

Ein falsch bereinigter Split sieht aus wie eine außergewöhnliche Rendite. Ein 2:1-Split, der in den historischen Kursen nicht korrekt abgebildet ist, erscheint als +100 % in einem einzigen Jahr. Der Screener sieht ein fantastisches Jahr; der Anleger sieht ein Datenartefakt.

```
def detect_split_anomalies(ticker: str,
                           threshold: float = 0.8) -> list[dict]:
    """
    Flags years where annual return exceeds threshold in
    either direction and cross-checks against known split
    dates from yfinance corporate actions data.

    Returns list of flagged years with context.
    """
```

Jedes Jahr mit einer Rendite über +80 % oder unter −80 % wird zur Prüfung markiert. Die Funktion prüft, ob in diesem Jahr ein bekannter Split stattfand. Wenn ja – plausibel. Wenn nein – mögliches Datenartefakt, manuell prüfen.

> *Das entfernt markierte Jahre nicht automatisch. Es macht sie sichtbar. Die Entscheidung, zu behalten oder auszuschließen, bleibt bei dir.*

Der Zombie-Detektor

Eine Zombie-Aktie wird technisch noch gehandelt, hat aber keine nennenswerte Kursaktivität. Geringes Volumen, Kurs bewegt sich kaum, faktisch illiquide. Solche Aktien bestehen den Ticker-Validator – die Daten existieren –, verschmutzen aber die Screener-Ausgabe, weil ihre Renditehistorie künstlich ist.

Die gefährlichere Variante ist der reaktivierte Zombie. Diese Aktien lagen jahrelang brach – Kurs flach bei oder nahe 1 $, minimales Volumen, kein echter Markt – und wurden dann plötzlich wieder gehandelt. Eine Aktie, die in einem Jahr von 1 $ auf 3 $ steigt, zeigt +200 %. Mathematisch korrekt. Praktisch bedeutungslos, und tödlich für jedes renditebasierte Ranking.

Japan Airlines ist der Lehrbuchfall. JAL meldete im Januar 2010 Insolvenz an und wurde kurz darauf vom Kurszettel genommen. Im September 2012, nach einer staatlich gestützten Restrukturierung, kehrte die Aktie an die Tokioter Börse zurück. Je nachdem, wie ein Datenanbieter die Lücke behandelt, zeigt die historische Kursreihe entweder einen sauberen Bruch oder näht die Kurse vor und nach der Insolvenz zusammen. So oder so sind die berechneten Renditen rund um die Wiederaufnahme extrem – und sagen nichts Brauchbares über JAL als Investment nach der Restrukturierung.

```
def detect_zombies(ticker: str,
                   price_floor: float = 2.0,
                   min_avg_volume: int = 100_000,
                   dormancy_threshold: float = 0.05) -> dict:
    """
    Detects two zombie patterns:

    1. Active zombie: trading but effectively illiquid
       -> avg daily volume below min_avg_volume

    2. Reactivated zombie: long dormancy period
       (price near floor, near-zero annual movement)
       followed by extreme return spike

    Returns classification and flagged periods.
    """
```

Zum reaktivierten Zombie konkret: Der Detektor sucht nach mehrjährigen Phasen, in denen die jährliche Kursbewegung unter der Ruheschwelle bleibt – weniger als 5 % in beide Richtungen –, gefolgt von einem Jahr mit einer überdimensionierten Rendite. Diese Kombination ist die Signatur.

Ticker	Muster	Markierter Zeitraum	Anmerkung
Aktie K	Aktiver Zombie	2019–2023	Ø Volumen: 8.200
JAL (9201.T)	Reaktivierter Zombie	Delisting 2010, Rückkehr 2012	Ruhephase + extremer Renditesprung
Aktie M	Aktiver Zombie	2021–2023	Kursspanne 0,80–1,10 $

Drei Aktien aus einem Universum von 80 markiert. Alle hatten den Ticker-Validator bestanden.

Ohne den Zombie-Detektor hätte JAL in jedem renditebasierten Screener nahe der Spitze gelegen – und nichts Brauchbares über das Unternehmen gesagt, wie es heute tatsächlich existiert.

Das eine, was kein Detektor ersetzt

Lass alle drei laufen. Behebe, was sie markieren. Dann schau dir die Ausgabe selbst an.

Das ist kein Disclaimer, der im Anhang vergraben ist. Es ist ein Kernbestandteil der Methode.

Ein Screener reduziert ein Universum von Hunderten Aktien auf eine Shortlist von Kandidaten. Das ist sein Job – und den macht er gut. Was er nicht kann: wissen, was du weißt. Er weiß nicht, dass ein Unternehmen auf der Liste gerade eine große Restrukturierung angekündigt hat. Er weiß nicht, dass ein Sektor regulatorischen Gegenwind hat, der sich in den Zahlen noch nicht zeigt. Du weißt es.

Der letzte Schritt ist immer eine manuelle Prüfung jedes Kandidaten, der es auf die Liste schafft. Keine stundenlange Recherche – ein Plausibilitätscheck. Aktuelle Nachrichten, letzte Berichte, alles, was dich innehalten lässt. Der Screener verdient sich deine Aufmerksamkeit. Die Entscheidung bleibt bei dir.

> *Ein Framework, das das Urteil vollständig ersetzt, ist kein Framework. Es ist eine Haftung.*

Teil 3

Der Screener

Der Pseudocode aus Teil 1 hat die Logik beschrieben. Die Datenqualitätsschicht aus Teil 2 hat die Eingabe bereinigt. Jetzt kommen beide zusammen.

Der Screener ist eine einzige Funktion mit konfigurierbaren Parametern. Sie nimmt eine validierte Ticker-Liste, holt die Renditehistorie, wendet den Goal-First-Filter an und liefert einen rangierten DataFrame. Die Struktur ist für jede Strategie dieselbe – was sich ändert, sind die Parameter.

Setup

Drei Bibliotheken decken alles ab, was dieses Framework braucht:

```
pip install yfinance pandas numpy
```

```
import yfinance as yf
import pandas as pd
import numpy as np
```

yfinance holt die Daten. pandas übernimmt Renditeberechnung und Filterung. numpy die Arithmetik. Keine weiteren Abhängigkeiten.

> Ein Wort zu yfinance, bevor du es installierst: Die Bibliothek ist ein unabhängiges Open-Source-Projekt, das Daten von Yahoo Finance abruft – ohne offizielle Schnittstelle und ohne Verbindung zu Yahoo. Was du mit den Daten tun darfst, regeln die Nutzungsbedingungen von Yahoo, nicht die Bibliothek. Für den eigenen Screener am eigenen Rechner ist das in der Regel unkritisch. Willst du Ergebnisse weitergeben, veröffentlichen oder geschäftlich nutzen, prüfe die Bedingungen vorher selbst – sie ändern sich, und dieser Guide kann sie nicht für dich lesen.

Jahresrenditen holen

yfinance liefert tägliche bereinigte Schlusskurse. Jahresrenditen musst du daraus berechnen – ein Jahresendkurs geteilt durch den Jahresendkurs des Vorjahres, minus eins.

```
def get_annual_returns(ticker: str,
                       years: int = 20) -> pd.Series:
    """
    Fetches adjusted closing prices and calculates
    calendar year returns.

    Returns pd.Series indexed by year (int),
    values as decimals e.g. 0.15 for +15%.
    """
```

Die Metriken in Teil 4 brauchen außerdem die Kursreihe selbst, nicht nur die Jahresrenditen. Dafür eine zweite, kleinere Funktion:

```
def get_prices(ticker: str,
               years: int = 20) -> pd.Series:
    """
    Fetches adjusted closing prices for the lookback period.

    Returns pd.Series indexed by date, chronological order.
    """
```

BEREINIGTE KURSE

yfinance liefert standardmäßig split- und dividendenbereinigte Kurse. Für die Outperformance-Strategie ist das, was du willst – Gesamtrendite inklusive Dividenden. Für einen reinen Kursvergleich übergib auto_adjust=False. Wisse, welche Variante du nutzt, bevor du die Ausgabe interpretierst.

KALENDERJAHRE

Die Funktion liefert eine Zahl pro Kalenderjahr – Januar bis Dezember. Das macht Trefferquote und Verlustjahrzählung intuitiv und zwischen Aktien direkt vergleichbar. Eine Aktie mit starkem vierten Quartal, aber schwachem Gesamtjahr zählt trotzdem als Verlustjahr. Das ist das korrekte Verhalten.

Die Kern-Screening-Funktion

Jeder Parameter hat einen Standardwert, aber keiner ist fest.

```
def screen(tickers: list[str],
           benchmark: str = '^GSPC',
           min_bias: float = 0.60,
           max_loss_ratio: float = 0.25,
           min_years: int = 10,
           years: int = 20) -> pd.DataFrame:
    """
    Applies goal-first screening logic to a validated
    ticker list.

    Parameters:
        tickers        - validated ticker list
        benchmark      - benchmark ticker (default: S&P 500)
        min_bias       - goal: min fraction of years beating benchmark
        max_loss_ratio - constraint: max fraction of years with negative return
        min_years      - constraint: minimum years of data required
        years          - lookback period in years

    Returns:
        pd.DataFrame sorted by avg_annual_return descending.
        Columns: ticker | bias_score | loss_year_ratio | avg_annual_return
    """
```

Die Funktion liefert nur Kandidaten, die Ziel-Schwelle und alle Constraints bestanden haben, rangiert nach durchschnittlicher Jahresrendite. Ausgeschlossene Ticker erscheinen in der Ausgabe nicht.

Wie die Ausgabe aussieht

Anonymisiertes Beispiel – 40 Ticker rein, 7 haben alles bestanden:

Ticker	Trefferquote (bias_score)	Verlustjahrquote	Ø Jahresrendite
Aktie A	0,75	0,10	+18,4 %
Aktie B	0,70	0,15	+16,1 %
Aktie C	0,65	0,20	+14,8 %
Aktie D	0,80	0,20	+13,9 %
Aktie E	0,60	0,25	+12,7 %
Aktie F	0,65	0,15	+11,3 %
Aktie G	0,60	0,20	+10,8 %

33 Ticker ausgeschlossen. 7 Kandidaten bleiben, rangiert nach durchschnittlicher Jahresrendite. Anonymisiertes Beispiel.

> *Das ist die Shortlist. Keine Kaufliste. Der nächste Schritt gehört dir.*

An ein anderes Ziel anpassen

Der Wechsel von Outperformance zu Dividende verlangt kein Neuschreiben des Screeners. Er verlangt, dass du änderst, was du holst und wonach du filterst.

Für einen Dividenden-Screener wechselt die Zielkennzahl von der Trefferquote zur Dividendenrendite. Die Constraints werden Verlustjahrquote und Ausschüttungsquote. Die Kernfunktion bleibt identisch. Die Signatur steht in Teil 5, Variante 1.

> *Hast du eine Version davon gebaut, hast du alle gebaut. Der schwerste Teil ist der erste – und den hast du bereits hinter dir.*

Teil 4

Metriken ergänzen

Der Screener rangiert Kandidaten nach durchschnittlicher Jahresrendite. Das reicht für eine brauchbare Shortlist. Aber eine Shortlist ist ein Ausgangspunkt, kein Ergebnis.

Metriken sind beschreibend: Sie sagen dir mehr über die Kandidaten, die bereits bestanden haben, damit deine manuelle Prüfung informiert ist statt blind. Drei Metriken decken das meiste ab, was zählt. Jede ist eine eigenständige Funktion, die eine Renditereihe nimmt und eine einzige Zahl liefert.

Metrik 1 — Durchschnittliche Jahresrendite

Im Screener bereits verwendet. Als eigenständige Funktion trotzdem sinnvoll, weil sie in jeder Strategievariante auftaucht.

```
def average_annual_return(annual_returns: list[float]) -> float:
    """
    annual_returns: list of decimal returns
                    e.g. [0.15, -0.12, 0.23]
    returns: arithmetic mean as decimal
             e.g. 0.087 for 8.7%
    """
    return sum(annual_returns) / len(annual_returns)
```

Jahr 1: +15 %. Jahr 2: −12 %. Jahr 3: +23 %. Durchschnitt: 8,7 %. Jedes Jahr wird als unabhängige Beobachtung behandelt. Das ist der richtige Ansatz, wenn du Jahresscheiben über verschiedene Aktien vergleichst – du misst die Konsistenz der jährlichen Performance, nicht das Endvermögen. Jedes Jahr steht für sich.

Metrik 2 — Maximaler Drawdown

Der maximale Drawdown misst den größten Rückgang vom Hoch zum Tief im beobachteten Zeitraum. Er beantwortet die Frage, die jeder Anleger irgendwann stellt: Was ist das Schlimmste, was diese Aktie getan hat – und hätte ich das durchgehalten?

```
def max_drawdown(prices: pd.Series) -> float:
    """
    prices: pd.Series of adjusted closing prices,
            chronological order

    returns: maximum drawdown as negative decimal
             e.g. -0.43 for 43% peak-to-trough decline
    """
```

Ein maximaler Drawdown von −43 % heißt: Irgendwann im Zeitraum saß ein Anleger, der am Hoch gekauft hatte, auf 43 % Verlust, bevor es sich erholte. Das ist keine Hypothese – es ist die gelebte Erfahrung jedes Einzelnen, der durchgehalten hat.

Der maximale Drawdown passt natürlich zur Verlustjahrquote. Die Verlustjahrquote sagt dir, wie oft eine Aktie auf Jahresbasis negativ war. Der maximale Drawdown sagt dir, wie schlimm die schlimmste Episode war. Beide zusammen geben ein viel vollständigeres Bild davon, wie es sich anfühlt, diese Aktie zu halten.

> *Eine Aktie mit niedriger Verlustjahrquote, aber hohem maximalem Drawdown ist eine, die meistens in Ordnung ist – bis sie es wirklich nicht ist.*

Metrik 3 — Benchmark-Trefferquote

Im Screener bereits als Zielvariable verwendet. Als eigenständige Metrik liefert sie mehr Auflösung – nicht nur, wie oft die Aktie die Benchmark geschlagen hat, sondern um wie viel in den Jahren, in denen sie es tat.

WARUM KALENDERJAHRE, KEINE ROLLIERENDEN FENSTER

Jedes Kalenderjahr hat seinen eigenen Marktcharakter – ein Sektor, der läuft, ein Makrothema, das dominiert, ein Zinsumfeld, das bestimmte Branchen begünstigt oder bestraft. Diese Themen folgen Kalenderjahren, nicht beliebigen 12-Monats-Fenstern.

Rollierende Fenster erzeugen eine bestimmte Verzerrung: Ein starkes Jahr, das etwa von Oktober bis September läuft, wird auf zwei rollierende Perioden aufgeteilt. Beide erhalten einen positiven Beitrag aus derselben zugrunde liegenden Performance. Die Trefferquote zählt es doppelt. Spiegelbildlich bei schwachen Jahren – ein schlechtes Jahr wird zu zwei negativen rollierenden Perioden. Kalenderjahre sind diskret. Jedes Jahr steht für sich. Ein gutes Jahr zählt einmal. Ein schlechtes Jahr zählt einmal.

```
def benchmark_bias(stock_returns: pd.Series,
                   benchmark_returns: pd.Series) -> dict:
    """
    stock_returns:     annual returns as pd.Series indexed by year
    benchmark_returns: annual returns as pd.Series indexed by year

    returns: {
        'bias_score': float,         # fraction of years beating benchmark
        'avg_outperformance': float  # avg excess return in years stock won
    }
    """
```

	Trefferquote	**Ø Outperformance**
Aktie A	0,70	+9,2 %
Aktie B	0,70	+4,8 %

Gleiche Konsistenz, andere Größenordnung. Die Trefferquote zeigt beide als gleichwertig. Die Metrik macht den Unterschied sichtbar.

Aktie A schlägt die Benchmark in den Jahren, in denen sie gewinnt, fast doppelt so deutlich. Die Screener-Ausgabe würde beide als gleichwertig zeigen. Diese Metrik sagt dir, welche mehr leistet.

Die Architektur sauber halten

Jede Metrik ist eine Funktion, die eine Rendite reihe nimmt und eine Zahl liefert. Funktionen mit einer einzigen Verantwortung sind leicht zu testen, leicht zu ersetzen und leicht zu kombinieren. Eine neue Metrik an die Screener-Ausgabe anzuhängen ist eine Zeile:

```
results['max_drawdown'] = results['ticker'].apply(
    lambda t: max_drawdown(get_prices(t))
)
```

Der Screener muss nicht wissen, welche Metriken du angehängt hast. Die Metriken müssen nichts über den Screener wissen. Sie sind unabhängig – und das heißt, du kannst das Framework erweitern, ohne zu zerstören, was bereits funktioniert.

> *Fang mit den drei Metriken oben an. Ergänze weitere, wenn du eine konkrete Frage hast, die sie nicht beantworten. Bau nur, was du brauchst.*

Teil 5

An dein Ziel anpassen

Screener, Datenqualitätsschicht und Metriken sind bisher um eine Strategie herum gebaut – Outperformance. Das ist Absicht. Ein Framework an einem konkreten Beispiel zu lernen ist leichter, als es abstrakt zu lernen.

Das Framework ist kein Outperformance-Screener. Es ist eine Struktur, die für jedes klar definierte Anlageziel funktioniert. Dieser Teil zeigt zwei Strategievarianten, jede aus denselben Bausteinen, jede nur mit geänderten Parametern und Metriken.

Das Muster, noch einmal

Jede Variante folgt derselben Abfolge:

Schritt	Aktion
1	Zielkennzahl definieren – wonach du rangierst
2	Constraints definieren – was einen Kandidaten disqualifiziert
3	Die richtigen Daten holen – Zielkennzahl + Constraint-Eingaben
4	Filter anwenden – dieselbe Kernlogik
5	Rangieren und prüfen – Ausgabe + manuelle Kontrolle

Zwischen Strategien ändern sich nur die Schritte 1 bis 3. Der Rest ist identisch.

Variante 1 — Dividende

Zielvariable: Dividendenrendite ≥ 3,5 % als Schwelle. Rangiert wird nach durchschnittlicher Gesamtrendite – Kursgewinn plus Dividende, wie in Guide 1.

CONSTRAINTS

Verlustjahrquote ≤ 0,25 – wie in der Outperformance-Variante. Ausschüttungsquote ≤ 0,90 – die Dividende muss aus Gewinnen finanziert sein, nicht aus Schulden oder Verkäufen von Vermögen.

```
def screen_dividend(tickers: list[str],
                    min_yield: float = 0.035,
                    max_loss_ratio: float = 0.25,
                    max_payout_ratio: float = 0.90,
                    min_years: int = 10) -> pd.DataFrame:
    """
    Dividend variant of the core screener.

    Goal: dividend yield above min_yield
    Constraints: loss year ratio, payout ratio
    Ranking: average annual total return, descending
    """
```

Das AT&T-Muster aus dem ersten Guide scheitert am Verlustjahr-Constraint. Vier negative Jahre in sechs (2017–2022) sind eine Quote von 0,67 – mehr als das Doppelte der Schwelle. Der Ausschüttungsquoten-Constraint fängt die Fälle ab, in denen die Verlustjahrhistorie akzeptabel aussieht, die Dividende aber künstlich aufrechterhalten wird.

Ticker	Dividendenrendite	Verlustjahrquote	Ausschüttungsquote	Ø Gesamtrendite
Aktie D	4,1 %	0,10	55 %	+11,2 %
Aktie B	4,8 %	0,15	71 %	+9,8 %
Aktie F	3,6 %	0,15	80 %	+9,1 %
Aktie C	4,3 %	0,20	68 %	+8,1 %
Aktie A	5,1 %	0,10	62 %	+7,4 %
Aktie E	3,8 %	0,20	74 %	+6,9 %

35 Ticker rein, 6 bestanden. Die höchste Dividendenrendite im gesamten Universum lag bei 8,9 % – sie scheiterte an Constraint eins. Rangiert nach Gesamtrendite, nicht nach Dividendenrendite. Anonymisiertes Beispiel.

Variante 2 — Low Volatility

Zielkennzahl: annualisierte Standardabweichung der Jahresrenditen – niedriger ist besser. Aufsteigend rangiert: die ruhigste Aktie oben.

CONSTRAINTS

Verlustjahrquote ≤ 0,25 – wie in den beiden anderen Varianten. Maximaler Einzeljahr-Drawdown ≤ 30 % – eine Aktie kann eine niedrige durchschnittliche Volatilität haben und trotzdem ein katastrophales Jahr; dieser Constraint fängt das ab.

```
def screen_low_volatility(tickers: list[str],
                          max_loss_ratio: float = 0.25,
                          max_single_drawdown: float = 0.30,
                          min_years: int = 10) -> pd.DataFrame:
    """
    Low volatility variant of the core screener.

    Goal metric: annualized standard deviation
    Constraints: loss year ratio, max single-year drawdown
    Ranking: std deviation ascending (lowest first)
    """
```

Die Standardabweichung ist ein Durchschnitt – sie glättet über einzelne Jahre. Eine Aktie mit überwiegend ruhigen Jahren und einem −45-%-Jahr kann trotzdem eine moderate Standardabweichung zeigen. Genau dieses eine Jahr ist das Ereignis, das Anleger zum schlechtesten Zeitpunkt verkaufen lässt. Der Constraint fängt es ausdrücklich ab, statt es im Durchschnitt verschwinden zu lassen.

Ticker	Ann. Std.-Abw.	Verlustjahrquote	Max. Einzeljahr-Drawdown
Aktie A	8,2 %	0,10	−11 %

Ticker	Ann. Std.-Abw.	Verlustjahrquote	Max. Einzeljahr-Drawdown
Aktie B	9,4 %	0,15	−14 %
Aktie C	10,1 %	0,10	−18 %
Aktie D	11,3 %	0,20	−22 %
Aktie E	12,0 %	0,15	−19 %
Aktie F	12,8 %	0,20	−27 %
Aktie G	13,4 %	0,25	−25 %
Aktie H	14,1 %	0,20	−29 %

Die volatilitätsärmste Aktie im gesamten Universum scheiterte an der Verlustjahrquote. Die zweitärmste hatte einen maximalen Einzeljahr-Drawdown von −38 %. Anonymisiertes Beispiel.

Deine eigene Variante bauen

Ist dein Ziel ein anderes – Value, Qualität, eine sektorspezifische Kennzahl –, ist der Bauprozess derselbe.

Definiere zuerst die Zielkennzahl. Dann frag: Welcher konkrete Fehlertyp würde dieses Ziel untergraben? Dieser Fehlertyp wird zum Constraint. Die Verlustjahrquote ist in allen drei gezeigten Varianten dabei; ob sie in deine gehört, entscheidet dein Ziel. Hol nur die Daten, die du brauchst. Lass die Kern-Screening-Funktion unverändert.

> *Das Framework ist nicht der Code. Das Framework ist das Denken. Der Code ist nur das, was passiert, wenn das Denken klar genug ist, um es aufzuschreiben.*

Fazit

Einmal bauen, immer nutzen

Das Framework in diesem Guide hat vier bewegliche Teile: eine Datenqualitätsschicht, die Probleme abfängt, bevor sie den Screener erreichen; eine Kern-Screening-Funktion, die deine Goal-First-Logik konsistent anwendet; eine Metrikschicht, die dir mehr über die Kandidaten sagt, die bestanden haben; und einen manuellen Prüfschritt, der dich in der Schleife hält.

Keiner dieser Teile ist für sich kompliziert. Nützlich werden sie, weil sie zusammenwirken – und weil sie dir gehören. Ein Screener, den du selbst gebaut hast, um deine eigenen Anlageziele herum, mit Constraints, die du aus Gründen gewählt hast, die du verstehst, ist ein grundlegend anderes Werkzeug als eines, das du heruntergeladen oder geerbt hast. Du weißt, was es tut und warum. Du weißt, was es nicht tut. Du kannst es anpassen, wenn sich deine Umstände ändern – wenn sich dein Einkommensbedarf verschiebt, deine Risikotoleranz wandert, das Leben etwas anderes von deinem Portfolio verlangt.

> *Das beste Anlagewerkzeug ist eines, das du vollständig verstehst. Alles andere ist eine Black Box mit einer hübschen Oberfläche.*

Dieser Guide hat die Outperformance-Strategie ausführlich behandelt und zwei weitere Varianten. Die Datenqualitätsschicht gilt für alle. Die Metriken gelten für alle. Die manuelle Prüfung gilt für alle. Willst du eine dritte Strategie hinzufügen, steht der Bauprozess in Teil 5. Willst du eine vierte Metrik, steht die Architektur in Teil 4.

Was dieser Guide – absichtlich – nicht behandelt hat, sind Portfoliokonstruktion, Positionsgrößen, Rebalancing und Ausführung. Das sind eigene Themen, jedes groß genug für eine eigene Behandlung. Der Screener sagt dir, welche Kandidaten eine Überlegung wert sind. Alles danach ist eine andere Entscheidung.

Noch etwas. Dieses Framework zu bauen braucht Zeit. Nicht weil der Code schwierig wäre, sondern weil das Denken dahinter deins sein muss – dein Ziel, deine Constraints, deine Risikotoleranz. Manche Anleger finden diesen Prozess reizvoll. Andere wollen ihn lieber überspringen und etwas haben, das funktioniert. Gehörst du zur zweiten Gruppe, melde dich über Gumroad.

Dieser Guide liefert ein Framework, keine Formel. Die Logik ist übertragbar – die Entscheidungen sind deine.

Ziel zuerst. Constraints danach. Einmal bauen, immer nutzen.

Der Rest ist Umsetzung.

Glossar

Methodenbegriffe wie in Goal First – Teil 1; dazu die Begriffe aus dem Code. Code-Namen in Klammern.

Adjustierter Kurs (adjusted close)

Historischer Kurs, rechnerisch um Splits und Dividenden bereinigt, damit Renditen über die Zeit vergleichbar sind. yfinance liefert ihn standardmäßig.

Benchmark-Trefferquote (bias_score)

Anteil der Kalenderjahre, in denen die Aktie die Benchmark (^GSPC) geschlagen hat. Zielvariable der Outperformance-Strategie; Schwelle hier 0,60.

Constraint

Binäre Ausschlussbedingung. Bestanden oder nicht; keine Teilpunkte. Jeder Constraint schützt vor einem Fehlertyp.

DataFrame / Series (pandas)

Tabellen- bzw. Spaltenstruktur in pandas. Der Screener liefert einen DataFrame mit einer Zeile je Kandidat.

Durchschnittliche Jahresrendite (avg_annual_return)

Arithmetisches Mittel der Kalenderjahr-Renditen. Ranking-Kennzahl der Outperformance- und der Dividendenstrategie.

Kalenderjahr-Rendite

Rendite 1. Januar bis 31. Dezember. Die Methode zählt in Kalenderjahren, nicht in rollierenden Fenstern – jedes Jahr zählt genau einmal.

Maximaler Drawdown (max_drawdown)

Größter Rückgang vom Hoch zum Tief im Zeitraum, als negative Dezimalzahl (−0,43 = 43 %).

Pseudocode

Logik in Programmform, aber ohne Syntax einer Sprache. Lesbar für jeden; Zwischenschritt vor dem Python-Code.

Split (stock split)

Aufteilung einer Aktie in mehrere (2:1 halbiert den Kurs). Nicht bereinigt sieht ein Split wie ein Kurssturz oder -sprung aus.

Standardabweichung, annualisiert

Maß für die Schwankungsbreite der Jahresrenditen. Zielkennzahl der Low-Volatility-Variante; niedriger ist besser.

Ticker

Börsenkürzel (SNAP, 9201.T). ^GSPC ist das Kürzel des S&P 500 bei Yahoo Finance.

Universum (universe)

Die Liste der Ticker, die gescreent wird – Indexmitglieder, ein Sektor, eine Watchlist.

Verlustjahrquote (loss_year_ratio)

Anteil der Kalenderjahre mit negativer Rendite. In allen gezeigten Strategien als Constraint verwendet; Standardwert im Code 0,25 (max_loss_ratio), frei wählbar.

yfinance / pandas / numpy

Die drei Python-Bibliotheken des Frameworks: Datenabruf von Yahoo Finance, Tabellenverarbeitung, Arithmetik.

Zombie-Aktie

Technisch gehandelte Aktie ohne echten Markt (Volumen, Kursbewegung). Reaktivierte Zombies erzeugen nach Ruhephasen extreme Scheinrenditen.

Über die Autorin

Leverage Thinking ist das Pseudonym von Laura Kruse-Stecher. Aktieninvestorin seit über zwanzig Jahren. Proprietäre Algorithmen zur Aktienauswahl im Live-Handel. Datenintegritäts- und Backtesting-Infrastruktur: Python, PostgreSQL/TimescaleDB. Ökonomin, geschäftsführende Gesellschafterin der Elcando GmbH. Die Praxis aus der Einleitung des ersten Guides wurde durch die Krise gebracht und später verkauft.

Fragen, Anmerkungen, Custom-Builds: über Gumroad.

Code & Updates

Der Code zu diesem Framework – Datenqualitätsschicht, Screener und Metriken aus Goal First in Python – liegt in einem öffentlichen Repository:

https://github.com/Elcando-GmbH/goal-first

Dort findest du auch Korrekturen und Aktualisierungen zu beiden Guides. Hinweise auf Fehler sind willkommen – über Gumroad oder direkt im Repository. Der Code steht unter der MIT-Lizenz.

Impressum

Autorin: Laura Kruse-Stecher, unter dem Pseudonym Leverage Thinking.

Impressum und Kontakt: leveragethinking.gumroad.com

Dieser Guide wurde unter Einsatz von KI-Werkzeugen für Recherche, Übersetzung und Textarbeit erstellt. Methode, Inhalte, Beispiele und alle Entscheidungen stammen von der Autorin.